

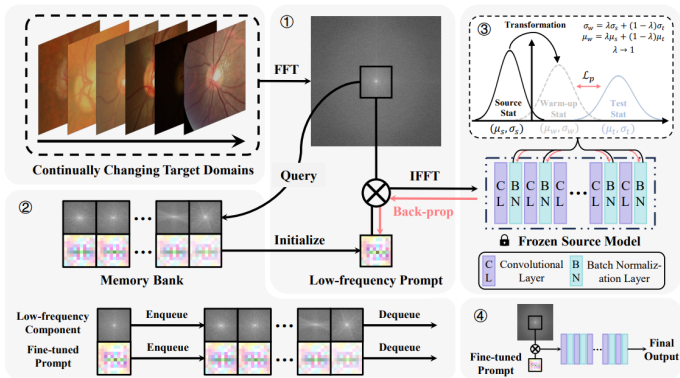
Frequency-Domain Visual Prompting for Test-Time Adaptation

Vuppula Hemanth Reddy
AI23BTECH11033

Mohammed Abdur Rehman
AI23BTECH11015

IIT Hyderabad
Computer Vision Project

Motivation-VPTTA



Problem Statement

- ▶ VPTTA performs test-time adaptation using image-specific prompts in the frequency domain
- ▶ Prompts are directly optimized during test time
- ▶ Adaptation relies on BatchNorm statistics

Limitations

- ▶ Finite FIFO memory discards older prompts, which may be useful when similar inputs reappear
- ▶ Dependence on BatchNorm restricts applicability to certain architectures
- ▶ Primarily focuses on low-frequency components

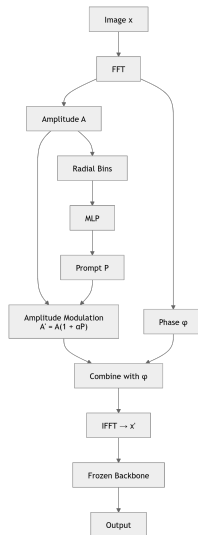
Problem statement-continued

Our Approach

- ▶ Replace finite memory with an MLP-based prompt generator
- ▶ Update generator during test time
- ▶ Produce input-dependent prompts from frequency features
- ▶ Use both low and high frequency information
- ▶ Applicable across different model architectures

Method Overview

- ▶ Convert image to frequency domain (FFT)
- ▶ Extract frequency features
- ▶ Generate prompt using MLP
- ▶ Modify amplitude
- ▶ Reconstruct and predict



Frequency Representation

- ▶ Input image: $x \in \mathbb{R}^{C \times H \times W}$
- ▶ Convert to frequency domain using Fourier Transform

$$\hat{x} = \mathcal{F}(x) \quad (1)$$

$$A = |\hat{x}|, \quad \phi = \angle \hat{x} \quad (2)$$

- ▶ Amplitude (A): magnitude of frequency components
- ▶ Phase (ϕ): spatial arrangement of structures

Key Insight

- ▶ Amplitude controls appearance (noise, blur, texture)
- ▶ Phase preserves structure (edges, shapes)

Radial Frequency Features

- ▶ Reduce amplitude map to radial features
- ▶ Divide frequency space into radial bins based on distance from center

$$e_{c,n} = \frac{1}{|R_n|} \sum_{(u,v) \in R_n} A_c(u, v) \quad (3)$$

- ▶ R_n : set of frequency locations in the n -th radial bin
- ▶ $e_{c,n}$: average amplitude in bin n for channel c
- ▶ Low-index bins $\rightarrow$ low-frequency components
- ▶ High-index bins $\rightarrow$ high-frequency components

Intuition

Captures how amplitude is distributed across different frequency bands

Prompt Generation

$$p = \text{MLP}(e) \quad (4)$$

- ▶ Input: radial frequency features $e \in \mathbb{R}^{C \times N}$
- ▶ MLP outputs coefficients $p \in \mathbb{R}^{C \times N}$
- ▶ Each coefficient corresponds to a radial frequency bin
- ▶ Coefficients are expanded to the full frequency domain
- ▶ Masks assign each frequency location to its corresponding bin

Frequency-Domain Adaptation

$$A' = A \cdot (1 + \alpha P) \quad (5)$$

- ▶ A : original amplitude, P : prompt expanded using masks
- ▶ α : learnable scaling parameter controlling update strength
- ▶ Prompt adjusts amplitude across frequency bins
- ▶ Phase ϕ is kept unchanged $\rightarrow$ preserves spatial structure

$$x' = \mathcal{F}^{-1}(A', \phi) \quad (6)$$

- ▶ Inverse FFT reconstructs the adapted image x'

Objective Function

$$L_{ent} = - \sum_k p_k \log p_k \quad (7)$$

- ▶ Entropy of model predictions on unlabeled test data
- ▶ Minimizing entropy $\rightarrow$ encourages confident predictions

$$L = L_{ent} + \lambda \|p\|^2 \quad (8)$$

- ▶ $\|p\|^2$: penalizes large prompt values
- ▶ Prevents excessive changes to the input
- ▶ Improves stability during test-time updates

Experimental Setup

- ▶ Backbone: ResNet-18 (kept frozen during adaptation)
- ▶ Datasets:
 - ▶ PACS – evaluates performance under domain shifts
 - ▶ CIFAR-10-C – evaluates robustness to corruptions
- ▶ Frequency Features: $N = 16$ radial bins
- ▶ Adaptation Setting:
 - ▶ Mini-batch: batch size = 32
 - ▶ Online: batch size = 1

Results: CIFAR-10-C

Corruption	S1				S2				S3				S4				S5			
	(1)	(2)	(3)	(4)	(1)	(2)	(3)	(4)	(1)	(2)	(3)	(4)	(1)	(2)	(3)	(4)	(1)	(2)	(3)	(4)
Gaussian Noise	73.82	72.18	81.78	80.81	52.57	52.32	70.53	69.72	35.92	35.89	56.64	55.24	30.40	29.86	52.58	50.64	25.92	25.78	40.34	46.63
Shot Noise	83.22	80.70	86.58	86.25	72.37	70.16	80.46	80.47	50.07	49.15	68.31	66.43	41.94	41.09	62.32	61.27	32.44	30.65	53.52	49.95
Impulse Noise	60.58	57.59	60.56	60.68	45.93	43.99	37.64	46.72	37.57	36.53	21.44	39.37	26.95	26.76	17.91	28.71	19.40	19.11	13.30	15.89
Defocus Blur	94.31	92.69	94.39	94.29	92.99	90.29	93.29	93.15	88.75	84.46	91.20	91.16	78.07	71.05	83.92	81.84	46.12	37.13	23.74	26.72
Glass Blur	54.92	49.00	63.39	63.80	55.53	50.04	62.70	63.52	60.62	55.92	65.62	67.44	41.91	36.59	13.97	19.22	45.84	41.80	18.72	33.23
Motion Blur	89.92	86.68	90.09	90.21	81.09	75.10	82.10	82.42	69.27	59.54	71.09	71.15	68.48	59.17	68.50	69.82	57.20	46.23	45.84	48.97
Zoom Blur	87.35	82.35	88.79	88.54	83.97	77.89	86.82	86.14	76.88	69.60	81.49	80.06	68.84	60.48	76.78	73.53	56.57	48.60	27.95	39.79
Snow	89.92	86.34	89.97	90.13	85.11	79.22	85.25	85.33	85.27	80.51	85.20	85.42	84.35	79.08	84.25	84.38	82.74	76.86	82.84	82.98
Frost	92.07	89.52	92.08	92.14	88.79	86.60	89.13	89.07	82.85	79.43	83.86	83.89	81.72	78.25	82.43	83.01	74.33	68.47	76.72	76.54
Fog	94.70	92.30	94.67	94.68	94.11	88.12	94.08	94.10	93.23	80.22	93.34	93.27	91.05	70.39	91.36	91.49	81.09	44.52	82.12	81.10
Brightness	94.41	93.03	94.39	94.38	94.03	92.46	94.00	94.06	93.65	92.23	93.66	93.65	92.95	92.24	92.99	92.92	90.72	90.24	90.67	90.72
Contrast	94.58	91.92	94.50	94.58	93.48	84.65	93.41	93.53	91.87	77.87	92.32	92.33	86.95	68.78	90.31	89.97	53.51	43.42	78.10	72.90
Elastic Transform	89.09	85.25	88.91	89.39	89.11	84.98	89.32	89.54	84.61	79.66	86.46	85.96	72.75	66.18	74.06	72.94	66.88	60.83	43.71	66.81
Pixelate	92.29	89.56	92.25	92.35	86.95	83.59	88.73	88.51	83.05	79.35	86.49	87.01	60.19	57.07	78.28	76.63	34.84	32.99	34.69	34.67
JPEG Compression	84.89	81.33	85.23	85.04	77.34	72.93	78.53	78.53	74.37	69.99	76.34	76.05	70.49	66.22	72.26	72.74	64.74	60.28	67.45	67.17
Speckle Noise	84.32	81.47	86.52	86.55	69.01	66.63	77.95	77.83	60.74	59.15	72.82	72.75	46.10	44.42	61.89	59.47	35.39	33.67	49.35	46.09
Gaussian Blur	94.28	92.67	94.30	94.33	88.81	84.05	91.38	91.24	78.84	71.16	86.63	84.38	63.68	53.36	73.12	69.95	31.77	25.57	20.63	25.03
Spatter	91.14	88.97	91.12	91.13	85.33	81.65	85.36	85.35	81.78	77.40	82.13	82.08	83.28	79.00	83.16	83.26	73.61	68.15	73.48	73.58
Saturate	92.83	87.06	92.81	92.80	90.10	84.13	90.45	90.40	93.82	92.62	93.85	93.89	91.32	90.11	91.39	91.42	87.16	85.36	87.91	87.52

- (1) Base Model
- (2) VPTTA (batch size = 1)
- (3) Ours (batch size = 1)
- (4) Ours (batch size = 32)

Table 1: CIFAR-10-C results across corruption types and severity levels

Results: CIFAR-10-C

- ▶ VPTTA struggles on corruption types like noise, where high-frequency components are affected
- ▶ We see that our method generally improves over the base model across most corruptions
- ▶ The improvements are more noticeable for noise and blur
 - ▶ like Gaussian noise, shot noise, motion blur, etc.
- ▶ Both $bs=1$ and $bs=32$ perform similarly, with $bs=32$ performing better in some cases and $bs=1$ in others

Results: PACS

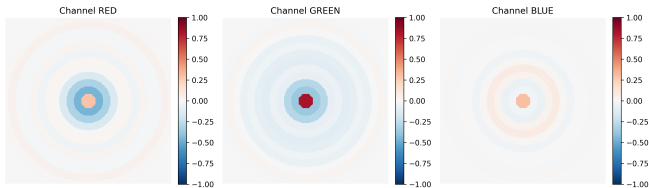
Domain	Base	VPTTA	Ours (bs=1)	Ours (bs=32)
Art	79.74	81.49	79.58	79.83
Cartoon	75.00	78.41	73.29	74.87
Photo	95.93	96.31	95.93	95.93
Sketch	73.04	76.46	72.43	73.30

Table: PACS Results across domains

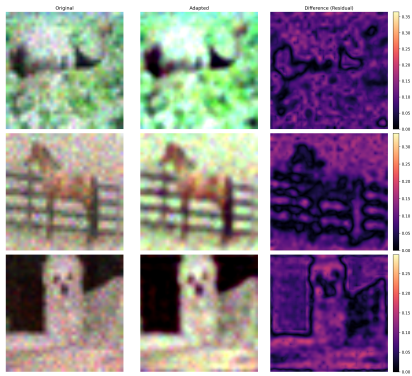
- ▶ VPTTA achieves the best performance across most domains
- ▶ Our method provides little to no improvement over the base model
- ▶ Online setting (bs=1) shows decrease in accuracy compared to bs=32 which implies overadaptation
- ▶ Suggests frequency-based adaptation is less effective for semantic domain shifts

Qualitative Results

Learned 2D Spectral Masks | shot_noise | Severity 3 | Alpha: 0.9265



Unified Spectral TTA Visualization | shot_noise | Severity 3



Conclusion

- ▶ Proposed a frequency-domain TTA method using MLP-based prompt generation
- ▶ Shows improvements on CIFAR-10-C, especially for noise and blur corruptions
- ▶ On PACS, gives small improvements over the baseline (bs=32), but remains below VPTTA
- ▶ Highlights limitations when handling semantic domain shifts

Key Insight

Frequency-based adaptation works well for corruption-based shifts, but is less effective for semantic variations

Thank You